What to eat! Requirements

Version 1.4 | October 28, 2026 | Juan Castro | Meta Global AI Developer Hackathon, Indie track

Overview

What to eat! is the middleman between you and your food decision. It is a chat: short questions, three real places on a map, and an app that gets better at reading you with every meal. Algorithms predict; Muse Spark (Meta Model API) handles language. Geoapify Places (OpenStreetMap data) supplies the restaurants; Supabase stores history and the learned profile.

Every recommendation traces to data. When evidence is thin, the app asks instead of assuming. Goals: efficient, cheap, accurate. The prototype costs $0 out of pocket.

Core rules

ID	Requirement	Phase
R1	Minimal UI with a glassy visual style (direction TBD). Every piece of text is as short as possible: one line per message. Recommendation cards show name, distance, price, and a one-line reason.	All
R2	Only places returned by the places API (Geoapify) are scored and shown. Muse never adds, removes, or reorders places.	1
R3	Log from day one: time, day of week, meal slot, answers, companions, budget level (asked), price level paid (if tapped), distance, the top 3, rejected picks with reasons, and the final choice. Phase 2 models depend on this data.	1

Functional requirements

Onboarding (taste quiz)

ID	Requirement	Phase
FR1	About 4 taps: favorite categories and hard no's (both from the ML2 list), typical budget per own meal. Answers form the quiz prior. Hard no's are a hard filter, never shown.	1
FR2	Quiz asks whether the user mostly eats solo or with people. If with people, the user adds companions by name, which fills the companions table.	1

Chat and questions

ID	Requirement	Phase
FR3	The app is a chat. One question per message, drawn from a pool: craving (category), budget, distance, who you're with. The distance question uses 3 chips: Walk, Short drive, Worth the drive.	1
FR4	Exactly 3 questions (3 messages) before the first round of recommendations. The ML6 confidence gate chooses which 3; Muse only phrases them. Answers the models can predict are pre-filled and editable. Cold start (Phase 1): the gate runs on the quiz prior.	1-2
FR5	Free-text answers are parsed by Muse into the same structured answers as quick-reply chips (ML7).	1

Recommendations

ID	Requirement	Phase
FR6	Geoapify Places search (catering categories) from the current location, using the search radius defined in FR-R2.	1
FR7	Each candidate is scored deterministically (ML5); the top 3 are shown, each with a one-line reason written by Muse from the actual evidence (ML7).	1
FR8	The top 3 appear as compact cards and pins on an Apple Map, with "Powered by Geoapify" and "© OpenStreetMap contributors" shown (NFR5). Tapping Directions opens Apple Maps.	1
FR9	The user confirms which place they chose, or none.	1
FR9b	After the first round, the conversation stays open as long as needed; refinement follows the ML10 loop.	1

Feedback

ID	Requirement	Phase
FR10	A local notification fires 2 hours after the last app activity, only once a decision has been made. New activity reschedules it.	1
FR11	"Went?" detection on app open: a foreground location check within 250 m of the picked place, between pick time and pick + 4 h, counts as a likely visit. If unsure, a pop-up asks if the user went to their last pick: Yes / No. Yes leads to "How was it?" with 5 stars, then one optional "About how much?" with $ / $$ / $$$ / $$$$. No leads to "Thanks!". No other feedback questions.	1
FR12	A preference setting for chat input: text (default) or voice. Voice uses Muse Voice Transcribe, then Muse parses the transcript into structured answers (ML7).	3

ID	Requirement	Phase
FR13	Went, stars, and the price tap update the inputs of the cuisine and place models (ML3, ML4). No free-text profile rewrite.	2

Background location is out of scope: it needs a development build and a paid Apple account.

Companions

ID	Requirement	Phase
FR14	Per companion: meal count, likely days of the week, and most likely categories together, derived from the same meal counts as ML3.	2
FR15	Selected companions are ML1 context: they change which questions are asked (ML6) and how places score (ML3).	2

Insights

ID	Requirement	Phase
FR16	Charts: price by meal slot (amount paid for backfill; price tap for live meals, else session budget level), most common category by meal slot, distance traveled, meals by companion, top categories.	2
FR17	A one-line Muse-written takeaway at the top of the Insights screen, phrased from computed stats only.	2

Prediction model

Algorithms predict; Muse only handles language. Every value marked "starting value" is tuned only on live data, never on the backfill.

ID	Requirement	Phase
ML1	Context features: meal slot (breakfast, lunch, dinner, late), weekday or weekend, companions (live data only for now).	1
ML2	Fixed category list, used by backfill, models, and place mapping: Burgers, Pizza, Mexican, Latin, Chinese, Japanese/Sushi, Korean, Thai, Vietnamese, Indian, Mediterranean, Italian, French, American/Diner, BBQ, Chicken/Wings, Sandwiches, Seafood, Salads/Bowls, Breakfast/Brunch, Coffee/Bakery, Smoothies/Juice, Dessert. Mapping is deterministic in two passes: first the Geoapify catering categories (from OSM cuisine tags), then name-keyword rules and a common-chains list (e.g. "taqueria" to Mexican, "pho" to Vietnamese, Chipotle to Mexican). Only places matching neither go to Other, which is shown but never predicted.	1

ID	Requirement	Phase
ML3	Layer 1, cuisine model: recency-weighted counts per context. Backs off to broader contexts when data is thin: this companion, then user overall, then quiz prior. Outputs a probability per category plus the number of meals behind it.	2
ML4	Layer 2, place model: return score from visits, recency, and 1-5 star ratings; 1-2 stars drop it sharply. New places are scored on category fit and distance fit. Price fit uses the user's own price data for that place; unknown = half.	2
ML5	Scoring: each real candidate is scored deterministically from ML3, ML4, price fit, and distance fit. OSM has no ratings or price levels, so neither is used. Top 3 = 2 familiar picks + 1 discovery slot (best candidate outside the user's usual categories). Phase 1 uses the quiz prior in place of ML3 and ML4.	1
ML6	Confidence gate: if the top prediction is weak or rests on too few meals, the app asks a question instead of assuming. This decides which of the 3 questions are asked. When price is uncertain, it may ask the budget question with the same 4 buttons ($ / $$ / $$$ / $$$$).	2
ML7	Muse is language only: parse free text into structured answers, write each pick's one-line reason from the actual evidence, run the conversation. Muse never reorders, removes, or adds places. One Muse call writes all 3 reasons; its output is checked against the candidate IDs.	1
ML8	Funnel logged per recommendation: shown, interested (card or directions tapped), rejected (with reason), picked, went, stars, price tap.	1
ML9	Evaluation: time-ordered replay on backfilled history (predict each meal using only earlier meals). Report cuisine and place top-3 hit rates against two baselines (always most frequent; random), with sample size stated.	2
ML10	Refinement loop: if the user isn't satisfied, each answer updates the session (constraints and model inputs) and the engine re-scores deterministically. Phase 1 re-runs with the answer only; Phase 2 picks the next question by expected information gain over the model's predictions. Rejected picks are logged as funnel events with the reason. After 3 rounds without a pick, show the best options so far plus one open question.	1-2

Starting values

Item	Starting value
Meal slot	Breakfast 05:00 to 10:59; lunch 11:00 to 15:59; dinner 16:00 to 20:59; late 21:00 to 04:59. Late meals after midnight count on the previous day's local date (and weekday).
Likely visit (FR11)	Within 250 m of the picked place, between pick time and pick + 4 h.
Usual categories (ML5)	Categories with at least 15% of the user's recency-weighted meals. Until data exists: quiz favorites.

Item	Starting value
Weak prediction (ML6)	Top probability under 0.4, or fewer than 5 meals behind it (matches FR-R3).
Price levels	$ = under $15; $$ = $15 to $30; $$$ = over $30 to $60; $$$$ = over $60 (per own meal).
Price fit	Compares levels: candidate at or below the budget level = full fit; one level above = half; two or more above = none; unknown = half. A place's known price = the user's most recent price tap or backfill amount for it.
Refinement cap (ML10)	3 rounds without a pick.
Distance chips (FR3)	Walk = 1,500 m; Short drive = 5,000 m; Worth the drive = 15,000 m.
Cold-start radius (FR-R3)	5 km.

Data

- meals is the source of truth for the models (backfill and live in one shape); recommendation_events is the funnel log (ML8).
- Price everywhere means what the user paid for their own meal. Backfilled meals keep the real amount, converted to a level with the table above. Live meals store the session budget level (what the user wanted) and the optional price tap (what it cost). Both are nullable, never guessed.
- places table: id (OSM type and id if Geoapify returns them, else the Geoapify place_id), name, coordinates, and our ML2 category. Geoapify allows storing results with no limits; attribution is shown wherever they appear (NFR5).
- Backfill: Juan's last 30 days of eating out, entered as CSV with columns date, meal_slot, place, category, price, companions. Companions are filled only when sure. Missing values stay null, never guessed.
- On import, each place name is resolved once to a Geoapify place so backfilled and live visits match. Ambiguous matches are confirmed by Juan.
- When the user's own backfill gives a category for a resolved place, that label overrides the ML2 mapping for that place.
- Backfill places that can't be resolved are stored with the CSV's name and category and flagged unresolved. Those meals count for the cuisine model, not for distance or place matching.
- Backfill tests Layers 1 and 2 by meal slot and day. Companion patterns and star-based return scores learn from live use only. Backfill has no location, so it does not feed the search radius.
- ML9 place hit rate: the candidate pool for each replayed meal is the set of places seen in earlier meals. A first visit to a new place counts as a miss; the share of new-place meals is reported separately. Parameters are fixed before replay, not tuned on it.

Adaptive search radius

The search area should grow more accurate to the user as usage increases. Values below are proposed starting points, to be tuned against real usage data.

ID	Requirement	Phase
FR-R1	Record the straight-line distance from the session location to the chosen place for every confirmed meal.	1
FR-R2	Radius for a new search = the recency-weighted 80th percentile of past travel distances in a similar context (ML1: meal slot, weekday or weekend, companions), clamped between 1 and 25 miles.	2
FR-R3	Cold start: a fixed 5 km radius until 5 confirmed meals exist for that context.	1-2
FR-R4	If fewer than 8 candidates come back, expand the radius by 1.5x, up to two times.	2

Non-functional requirements

ID	Requirement	Phase
NFR1	Runs on iPhone through Expo Go; no App Store build required.	1
NFR2	Backend deployed on a free tier by the end of Phase 1 (Vercel Hobby; Render free with a keep-alive ping as fallback), so the app works away from home Wi-Fi.	1
NFR3	API keys live only on the backend; every API request requires a valid Supabase JWT.	1
NFR4	First round of recommendations in under about 8 seconds after the third answer: one places search and one Muse call per round.	1
NFR5	Compliance: no nutrition, calorie, health, or diet claims anywhere. "Powered by Geoapify" and "© OpenStreetMap contributors" (linked to openstreetmap.org/copyright) shown wherever results appear. Everything presented was built Oct 26 to Nov 4; the backfill is pre-existing personal data, labeled as such in the demo and README.	All
NFR6	$0 out of pocket: free tiers only, no card on file (Geoapify free, Vercel Hobby, Supabase free, Expo Go).	All

Architecture

Mobile: Expo (React Native, TypeScript). **Backend:** FastAPI (Python, uv) on Vercel Hobby. **Places:** Geoapify Places API (OpenStreetMap data, ODbL). **Map:** Apple Maps via react-native-maps. **Prediction:** deterministic scoring and models in the backend (ML3 to ML6, ML10). **AI:** Muse Spark via Meta Model API for language only (ML7); Muse Voice Transcribe for

voice input. **Data and auth:** Supabase (Postgres).

Tables: profiles (linked to Supabase auth users), taste_profiles, companions, sessions, session_companions, messages, meals (source = backfill or live), meal_companions, categories, places, recommendation_events. Went, stars, and the price tap are funnel events (ML8) and are also kept on the meal; there is no separate feedback table.

Phases and deliverables

Phase	Scope	Due
1	**Core loop** (Tue Oct 27 to Thu Oct 29). Deliverable: the app in daily use on iPhone by Thursday night. Quiz, chat with 3 questions, scored top 3 on a map with Muse reasons, refinement loop (ML10 v1), confirm, Went? + stars + price tap with notification, full logging and funnel events, deployed backend.	Oct 29
2	**Learning and insights** (Fri Oct 30 to Sun Nov 1). Deliverable: the app visibly adapts to real data. Backfill import, cuisine and place models, confidence gate with information-gain questions, offline evaluation, adaptive radius, companion context, Insights charts.	Nov 1
3	**Voice and demo** (Mon Nov 2 to Wed Nov 4). Deliverable: project submitted by 9:00 PM ET on Nov 4. Voice input for chat answers, edge cases, polish, README with evaluation results, demo video.	Nov 4

Risks

- Phase 1 is the tightest phase and includes a cloud deploy. Every day it slips removes a self-use day, which is the core of the demo. First cut if needed: ML10 information gain stays in Phase 2; the Phase 1 loop only re-runs with the answer.
- Voice is deliberately Phase 3. If time runs out, text input still delivers the full product.
- 30 days of backfill is roughly 30 to 60 meals. Evaluation results carry wide uncertainty and are reported with sample size, not as proof.
- OSM cuisine tags are often missing. The name-keyword and chains pass (ML2) reduces this; places matching neither fall to Other and score lower on category fit.
- Free-tier cold starts can eat into the 8 second budget (NFR4). Cold-start latency is measured at deploy.
- The radius thresholds (80th percentile, 1 to 25 miles, 8 candidates) and all starting values above are untested guesses, to be tuned after 2 to 3 days of live data.

Changes from v1.3

- Tables: users renamed profiles; meal_companions added (backfill companions, FR14, FR16); feedback table dropped (covered by ML8 events and meals).
- FR-R3: cold start uses a fixed 5 km radius (the quiz has no distance question).
- FR3: distance question uses 3 chips: Walk 1,500 m, Short drive 5,000 m, Worth the drive 15,000 m.
- Meal slots: late = 21:00 to 04:59 on the previous day's date; breakfast starts at 05:00.
- Price levels: boundaries made exact ($30 is $$, $60 is $$$).
- Data: meals is the source of truth for the models; recommendation_events is the funnel log.

Changes from v1.2

- ML2: added French and Smoothies/Juice (23 categories); user's backfill label overrides the mapping.

Changes from v1.1

- Geoapify Places (OpenStreetMap) replaces Google Places. Results may be stored; the places table keeps id, name, coordinates, and ML2 category.
- New NFR5 (compliance: no health claims, Geoapify and OSM attribution, built during the event, backfill disclosed) and NFR6 ($0, no card).
- FR8: cards plus Apple Map pins; Directions opens Apple Maps.
- Price: live meals store the session budget level and an optional price tap after stars (FR11). ML6 may ask the budget question when price is uncertain.
- ML4, ML5: no ratings or price levels from the provider; new places scored on category fit and distance; price fit from the user's own data, unknown = half.
- ML2: Salads/Healthy renamed Salads/Bowls; mapping from Geoapify categories, then name keywords and a chains list.
- New ML10 refinement loop; ML8 adds rejected (with reason) and price tap.
- Data: unresolved backfill places stored with the CSV's name and category, used by the cuisine model only.
- NFR2: free hosting on Vercel Hobby, Render free as fallback.